

Document Number: p2927r0

Date: 2023-06-16

Target: EWG, LEWG, LEWGI

Reply to: gorn@microsoft.com

Inspecting exception_ptr

Abstract

Provide facility to observe exceptions stored in `std::exception_ptr` without throwing or catching exceptions.

Introduction

This is a followup to two previous papers in this area:

Date	Link	Title
Feb 7, 2018	https://wg21.link/p0933	Runtime introspection of exception_ptr
Oct 6, 2018	https://wg21.link/p1066	How to catch an exception_ptr without even try-ing

These papers received positive feedback. In 2018 Rapperswil meeting, EWG expressed strong desire in having such facility:

Does EWG want a non throwing mechanism to get to the exception held by exception_ptr even if the performance was the same

SF	F	N	A	SA
16	8	1	0	0

LEWG looked at this at SanDiego 2018 and encouraged to come back after addressing the following points:

- Remove the handle interface.
- Combine with P0933R0.
- Demonstrate field and implementation experience.
- Add wording
- Reach out to implementors and confirm this is implementable.

This paper brings back `exception_ptr` inspection facility in a simplified form addressing the earlier feedback.

Proposal at a glance

We introduce a single function `try_cast` that takes `std::exception_ptr` as an argument `e` and returns a pointer to an object referred to by `e`.

```
template <typename T>
const std::remove_cvref_t<T>*
try_cast(const std::exception_ptr& e) noexcept;
```

If `exception_ptr` is not empty and `std::remove_cvref_t<T>` is the type of the stored exception `E` or its unambiguous base, a const pointer to the stored exception is returned; otherwise `nullptr` is returned.

Example:

Given the following error classes:

```
struct Foo {
    virtual ~Foo() {}
    int i = 1;
};
struct Bar : Foo, std::logic_error {
    Bar() : std::logic_error("This is Bar exception") {}
    int j = 2;
};
struct Baz : Bar {};
```

The execution of the following program

```
int main() {
    const auto exp = std::make_exception_ptr(Baz());
    if (auto* x = try_cast<Baz>(exp))
        printf("got '%s' i: %d j: %d\n", typeid(*x).name(), x->i, x->j)
    if (auto* x = try_cast<Bar>(exp))
        printf("got '%s' i: %d j: %d\n", typeid(*x).name(), x->i, x->j)
    if (auto* x = try_cast<Foo>(exp))
```

```

        printf("got '%s' i: %d\n", typeid(*x).name(), x->i);
    }

```

results in this output:

```

got '3Baz' what:'This is Bar exception' i: 1 j: 2
got '3Baz' what:'This is Bar exception' i: 1 j: 2
got '3Baz' i: 1

```

See implementation for GCC and MSVC using available (but undocumented) APIs

<https://godbolt.org/z/ErePMf66h>.

Discussion

Let's look at existing facilities in the library that serve somewhat similar purpose:

any_cast:

```

template<class T> T any_cast(const any& operand);
template<class T> T any_cast(any& operand);
template<class T> T any_cast(any&& operand);

template<class T> const T* any_cast(const any* operand) noexcept;
template<class T> T* any_cast(any* operand) noexcept;

```

Last two overloads perform inspection of a stored value and give a pointer to it if the typeid matches, otherwise return nullptr.

The first three, throw bad_any_cast exception if the stored value does not match the requested type.

get/get_if:

```

// get forms that takes a reference and throw if the type is not active
template<class T, class... Types> T* get_if(variant<Types...>* pv) noexcept
template<class T, class... Types> const T* get_if(const variant<Types...>* pv) noexcept

```

Similarly to any_cast, get and get_if offer throwing versions that take a reference to a variant and a non-throwing that take a variant by pointer and return a pointer to the value of the requested type or nullptr if it is not an active alternative.

exception_ptr:

Q: Should we follow the the examples above and have two flavors of exception_ptr inspection, one that throws if the requested type E is not stored and one that does not.

A: No. The goal of this facility is to provide a way to inspect a stored exception value without relying on throwing exception.

Q: Non-throwing versions of dynamic_cast, any_cast and get_if all take a pointer to a value to be inspected, should this API take the pointer as well?

A: dynamic_cast and any_cast relies on a distinction on how the value is passed (by pointer or by reference) to select what semantic to provide. In our case, we always perform a conditional access, thus, requiring a pointer would be a needless syntactic noise.

Q: The variant and any inspection APIs offers a version that allows both const and non const access to the stored value. Should we do the same for exception_ptr?

A: No. N4928/[propagation]/7 states that:

For purposes of determining the presence of a data race, operations on exception_ptr objects shall access and modify only the exception_ptr objects themselves and not the exceptions they refer to.

Offering the API flavor that allows mutation of the stored exception has potential of injecting a data race. Cpp core guidelines recommend catching exception by const reference.

In this paper we only offer a try_cast API that offer an ability to inspect, but not modify stored exception. If in the future, we will discover an important use case that requires changing the stored exception a different API with lengthier and less convenient name, say try_cast_mutable or something along those lines can be added.

The desire is to offer the simplest name for the most common use case that is a good default for the majority of the users.

Similarly, we chose to specify the desired type in the simplest possible form, i.e:

```
auto* x = try_cast<const int*>(eptr); // no
auto* x = try_cast<int*>(eptr);      // no
auto* x = try_cast<int>(eptr);       // yes
```

Thus, the thought process above led to the following API shape:

```
template <typename T>
const std::remove_cvref_t<T>*
try_cast(const std::exception_ptr& e) noexcept;
```

Q: Could we reuse `any_cast` name?

A: No. It encodes `any` in its name strongly hinting it is meant to be used with `std::any`.

Q: Maybe `get_if` would work?

A: Possibly, but, it has slightly different semantics. Unlike `any_cast`, `dynamic_cast` and offered here `try_cast` have `cast` in its name, implying that it does not have to be exact match, whereas `get_if` expects the type to be exact match from the list of variant alternatives.

It is possible that in the future, we will have a unifying facility (pattern matching, for example) that would allow uniform access across variant, `any`, `exception_ptr` and other types. Finding such facility is out of scope of this paper.

Q: Why `const std::remove_cvref_t<T>*` in the return value? Could it be just `T`?

A: The intent here is to make sure that people who habitually write `catch (const E& e) { ... }` can continue doing it with `try_cast<const E&>`, as opposed to getting a compilation error. This is an error from the category: The compiler/library knows what you mean, but, will force you to write exactly as it wants.

Other names considered but rejected

- `cast_exception_ptr<E>`
- `cast_exception<E>`
- `try_cast_exception_ptr<E>`
- `try_cast_exception<E>`
- `catch_as<E>`

Pattern matching

We expect that `try_cast` will be integrated in the [pattern matching facility](#) and will allow inspection of `exception_ptr` as follows:

```
inspect (eptr) {
    <logic_error> e => { ... }
    <exception> e => { ... }
    nullptr => { puts("no exception"); }
```

```
__ => { puts(some other exception"); }  
}
```

Implementation

GCC, MSVC implementation is possible using available (but undocumented) APIs <https://godbolt.org/z/ErePMf66h>. Implementability was also confirmed by MSVC and libstdc++ implementors.

A similar facility is available in Folly and supports Windows, libstdc++, and libc++ on linux/apple/freebsd.

<https://github.com/facebook/folly/blob/v2023.06.26.00/folly/lang/Exception.h>
<https://github.com/facebook/folly/blob/v2023.06.26.00/folly/lang/Exception.cpp>

Implementation there under the names: folly::exception_ptr_get_object
folly::exception_ptr_get_type

Extra constraint imposed by MSVC ABI: it doesn't have information stored to do a full dynamic_cast. It can only recover types for which a catch block could potentially match. This does not conflict with the try_cast facility offered in this paper.

Usage experience

A version of exception_ptr inspection facilities is deployed widely in Meta as a part of Folly's future continuation matching.

Proposed wording

TBD

Acknowledgments

Many thanks to those who provided valuable feedback, among them: Aaryaman Sagar, Arthur O'Dwyer, Jan Wilmans, Joshua Berne, Lee Howes, Michael Park, Peter Dimov, Ville Voutilainen, Yedidya Feldblum.

References

<https://godbolt.org/z/ErePMf66h> (gcc and msvc implementation)

<https://wg21.link/p0933> Runtime introspection of exception_ptr

<https://wg21.link/p1066> How to catch an exception_ptr without even try-ing

<https://wg21.link/p1371> Pattern Matching